Capitulo 3. Registro de cambios con Git

1. Introducción

El registro de cambios es una de las funciones principales de Git. Permite guardar de forma ordenada las modificaciones realizadas en los archivos de un proyecto y asociarlas a un mensaje que explique que se hizo.

Este proceso se realiza principalmente con los comandos git add y git commit. EL primero prepara los cambios y el segundo los guarda permanentemente en el historial del repositorio.

2. El área de preparación

Antes de confirmar el cambio, Git utiliza un espacio llamado área de preparación o stating area. En esta área se seleccionan los archivos o modificaciones que serán incluidos en el próximo commit.

Esto permite tener un mayor control sobre el historial de ese proyecto, ya que no es necesario confirmar todos los archivos al mismo tiempo. Por ejemplo, se pueden preparar únicamente los cambios relacionados con un capítulo del tutorial y dejar otros archivos para un commit posterior.

3. Revisar los cambios con git status

El comando git status permite revisar el estado actual del repositorio antes de agregar o confirmar cambios.

```
Git status
```

Si se crea un archivo nuevo llamado cambios.txt, Git lo mostrara como un archivo sin seguimiento:

```
Untracked files:
        cambios.txt
```

Esto significa que el archivo existe en la carpeta, pero todavía no ha sido agregado al control de revisiones.

4. Agregar archivos con git add

El comando git add permite preparar archivos para ser confirmados. Para agregar un archivo especifico, se utiliza:

```
Git add cambios.txt
```

También es posible agregar varios archivos al mismo tiempo:

```
Git add capitulo—1.txt
Capitulo-2.txt
```

Si se desea agregar todos los archivos nuevos y modificados de la carpeta actual, se puede utilizar:

```
Git add .
```

El punto indica que se deben preparar todos los cambios ubicados en el directorio actual y sus subcarpetas.

Después de ejecutar git add se pueden verificar nuevamente el estado con:

```
Git status
```

Git mostrara los archivos en color verde, indicando que están preparados para ser confirmados.

5. **Confirmar cambios con git commit**

   El comando git commit guarda en el historial los cambios que fueron preparados con git add. Cada confirmación debe incluir un mensaje claro y descriptivo.
   La forma mas comun de utilizarlo es:

   Git commit –m “mensaje descriptivo del cambio”

   Por ejemplo:

   Git commit –m “agrega el capítulo 1 sobre introducción a git”

   También se puede usar:

   Git commit –m “agrega el capítulo 2 sobre los primeros pasos con git”

   Un buen mensaje de confirmación debe explicar brevemente el propósito del cabio. Por ejemplo, en lugar de escribir “cambios”, es preferible escribir “agrega el capítulo 3 sobre el registro de cambios en Git”.

6. **Ejemplo completo**

   El siguiente ejemplo muestra el flujo básico para registrar un cambio en un archivo de texto:

   ```
   git status
   git add cambios.txt
   git commit -m "Agrega archivo de registro de cambios”
   git status
   ```

   Primero se revisa el estado del repositorio. Luego se prepara el archivo con git add, se guarda el cambio con git commit y, finalmente, se verifica que el archivo haya sido confirmado correctamente.

7. **Buenas prácticas en los mensajes**

   Para mantener un historial claro, se recomienda:

   - Escribir mensajes breves, claros y descriptivos
   - Utilizar el presente: “agrega", "modifica” o “actualiza”
   - Realizar un commit por cada cambio o grupo de cambios relacionado.
   - Documentar cuando se agrega, modifica o corrige un archivo.

   Estas prácticas facilitan la revisión del historial y permiten comprender la evolución del proyecto.

8. **Referencia**

Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. Disponible en: https://git-scm.com/book/es/v2